
Parliament[™] User Guide



Ian Emmons

June 16, 2026

*Copyright © 2001-2026,
RTX BBN Technologies, Inc.
All rights reserved.*

Typeset using *Georgia*, *Verdana*, *Menlo*, and *TeX Gyre Bonum Math*

Contents

1	Introduction to Parliament™	1
1.1	Background	1
1.2	Parliament Architecture	2
2	Deploying and Using Parliament™	4
2.1	Deploying a Parliament Container	5
2.2	Deploying a Parliament Server	6
2.2.1	Installing for the First Time	6
2.2.2	Upgrading an Existing Installation	10
2.2.3	Usage	11
2.2.4	Securing a Parliament Server	16
2.3	Configuring Parliament	19
2.3.1	Common Configuration Tasks	19
2.3.2	Parliament's Startup Script	20
2.3.3	Parliament's Main Configuration File	21
2.3.4	Parliament's Logging Configuration	27
2.3.5	Parliament's Servlet Container Configuration	28
2.4	Parliament-Specific Features	29
2.4.1	Reserved Predicates	29
2.4.2	Reification	30
2.4.3	Configuring and Using Indexes	30
2.5	Parliament in Other Servlet Containers	30
2.6	Using Parliament In-Process	32
2.7	The ParliamentAdmin Utility	35
2.8	Troubleshooting	35
3	Building Parliament™	39

CONTENTS

3.1	Platforms and Prerequisites	39
3.2	Building RocksDB	41
3.2.1	Building RocksDB on Windows	41
3.2.2	Building RocksDB on Macintosh	41
3.2.3	Building RocksDB on Linux	44
3.3	Building the Boost Libraries	45
3.3.1	Building Boost on Windows	46
3.3.2	Building Boost on Macintosh	47
3.3.3	Building Boost on Linux	48
3.4	Configuring Boost.Build	49
3.5	Building Parliament Itself	51
A	Glossary	54
B	Bibliography	56

List of Figures

1.1	Layered Parliament Architecture	3
2.1	Issuing a SPARQL select query with Jena 3	13
2.2	Issuing a SPARQL-Update request with Jena 3	13
2.3	Inserting RDF with Jena 3	14
2.4	Using Parliament Via a Remote Jena Model	15
2.5	Creating a Jena Model backed by Parliament	33
2.6	Using a Jena Model backed by Parliament	34

List of Tables

2.1	Parliament Container Connection URLs	5
2.2	Supported Platforms	6
2.3	Parliament Server Connection URLs	12
3.1	Supported Platforms and Compilers	40
3.2	Supported Linux Flavors	48
3.3	Possible Values of BOOST_TEST_LOG_LEVEL	50
3.4	Boost.Build Search Paths for Configuration Files	50

Chapter 1

Introduction to ParliamentTM

ParliamentTM is a high-performance semantic graph store (also called a triple store) and reasoner designed for the Semantic Web.¹ RTX BBN Technologies (BBN) developed Parliament initially under the name DAML DB² under Defense Advanced Research Projects Agency's (DARPA's) DARPA Agent Markup Language (DAML) program. BBN later extended Parliament for internal use in its R&D programs and released Parliament as an open source project under the BSD license³ on SemWebCentral⁴ in 2009. In 2018, BBN migrated the Parliament open source project to GitHub⁵ under the same license.

ParliamentTM is a trademark of RTX BBN Technologies. It is so named because a group of owls is properly called a *parliament* of owls.

1.1 Background

The Semantic Web employs a different data model than a relational database. A relational database stores data in tables (rows and columns)

¹<http://www.w3.org/2001/sw/>

²<http://www.daml.org/2001/09/damldb/>

³<http://opensource.org/licenses/bsd-license.php>

⁴<http://parliament.semwebcentral.org/>

⁵<https://github.com/raytheonbbn/parliament>

while Resource Description Framework (RDF)⁶ represents data as a directed graph of ordered triples of the form (subject, predicate, object). Accordingly, a Semantic Web data store is often called a semantic graph, triple store, knowledge base, or graph store.

A relational database can store a directed graph, and some graph stores are in fact implemented as a thin interface layer wrapping a relational database. However, the query performance of such implementations is usually poor. This is because the only straightforward way to store the graph with the required level of generality is to use a single table to store all the triples, and this schema tends to defeat relational query optimizers.

Early in the Semantic Web's evolution, BBN encountered exactly this problem, and so the graph store we now call Parliament was born. The goal of Parliament was to create a storage mechanism optimized specifically to the needs of the Semantic Web, and the result was a dramatic speed boost for BBN's Semantic Web programs. Since its initial conception, Parliament has served as a core component of several projects at BBN for a number of U.S. Government customers.

1.2 Parliament Architecture

Parliament combines customized versions of the Web interface and query processor of Jena⁷ with a high-performance storage engine and an innovative storage layout to deliver a complete triple store solution that is compatible with the RDF, Web Ontology Language (OWL),⁸ and SPARQL Protocol and RDF Query Language (SPARQL)⁹ standards from the World Wide Web Consortium (W3C) [3].

In addition, Parliament includes a high-performance rule engine, which applies a set of inference rules to the directed graph of data in order to derive new facts. This enables Parliament to automatically and transparently infer additional facts and relationships in the data to enrich query results. Parliament's rule engine currently implements all of the inference rules of

⁶<http://www.w3.org/RDF/>

⁷<https://jena.apache.org/>

⁸<http://www.w3.org/2007/OWL/>

⁹http://www.w3.org/2009/sparql/wiki/Main_Page

RDF Schema (RDFS) plus selected elements of OWL RL.

Figure 1.1 depicts the layered architecture of Parliament. The storage layer of Parliament is written in C++, while the remainder is Java code. Integrated with the Jena query processor are a number of useful extras, such as support for named graphs, accelerated reification support, and temporal, geospatial, and numerical indexes [2, 1].

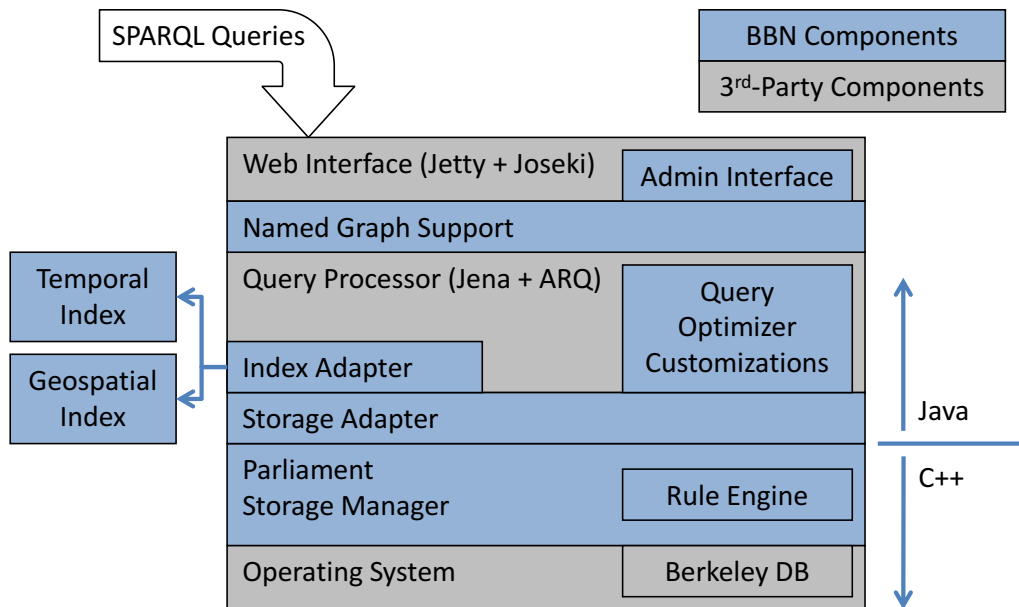


Figure 1.1: Layered Parliament Architecture

Chapter 2

Deploying and Using ParliamentTM

There are several ways to use Parliament. By far the most common is as a server application that exposes a SPARQL endpoint. Parliament's Docker image and its more traditional binary distribution directly support this usage, running Parliament within the included Jetty servlet engine. See Sections 2.1 and 2.2 for instructions.

Parliament can also be used with other servlet engines such as Tomcat and Glassfish (see Section 2.5) or as an in-process library (Section 2.6).

Pre-built binaries for common platforms are available on the Parliament Web site.¹ You can also build Parliament yourself (see Chapter 3).

This chapter starts with instructions for running a Parliament container (Section 2.1) and for installing a Parliament server (Section 2.2), followed by a discussion of the Parliament configuration file (Section 2.3). Then comes a detailed discussion of each of the less common deployment scenarios. The chapter concludes with a discussion of Parliament's utilities and some troubleshooting hints (Sections 2.7 and 2.8, respectively).

¹<https://github.com/raytheonbbn/parliament/releases>

2.1 Deploying a Parliament Container

Parliament's Docker image can be found on Docker Hub here:

```
idemmons/parliament:«version»
```

To run the container, use this command (with line breaks added for readability):

```
docker run -d -p 80:8089 --name=parliament-«version»  
  --mount type=volume,src=parliament-data,dst=/var/parliament-data  
  --env PARLIAMENT_JAVA_HEAP_SIZE=512m  
  idemmons/parliament:«version»
```

As given here, the mount option will create a local Docker volume called `parliament-data` to contain Parliament's data files and logs. Using variations of this option, you can cause the container to store the data files and logs in a broad variety of locations and file systems, such as on an external storage array. (See the Docker documentation for more information.)

Alternately, Docker can store Parliament's data files and logs in a local directory on the container's host with this syntax:

```
docker run -d -p 80:8089 --name=parliament-«version»  
  --mount type=bind,src=/host/directory,dst=/var/parliament-data  
  --env PARLIAMENT_JAVA_HEAP_SIZE=512m  
  idemmons/parliament:«version»
```

The two environment variables are optional. If they are left out, the default values used will be the values shown in the command line above. (See Sections 2.3.2 and 2.3.3 for discussions of these settings.) Table 2.1 lists the endpoints exposed by the running container, as discussed in Section 2.2.3.

HTTP interface:	<code>http://«host»/parliament</code>
SPARQL endpoint:	<code>http://«host»/parliament/sparql</code>
Bulk loader:	<code>http://«host»/parliament/bulk</code>

Table 2.1: Parliament Container Connection URLs

2.2 Deploying a Parliament Server

A Parliament distribution is a compressed archive containing a ready-to-run Parliament server, with the following naming convention:

`parliament-x.y.z-platform.zip`

Here *x.y.z* is the Parliament version number and *platform* indicates the platform, as shown in Table 2.2.² If you are building Parliament yourself

<i>Platform</i>	<i>Description</i>	<i>Binaries</i>
mac	Mac OS Catalina and newer	64-bit
rhel8	Red Hat Enterprise Linux 8	64-bit
ubuntu20	Ubuntu Linux 20, LTS	64-bit
win-64	Windows 10 and newer	64-bit
win-32	Windows 10 and newer	32-bit

Table 2.2: Supported Platforms

according to the instructions in Chapter 3, then these archives appear in the `target/distro` directory at the end of the build process.

As shown in Table 2.2, on Windows there is a choice between 32-bit and 64-bit builds. Whenever possible, use the 64-bit version, because 32-bit Parliament is limited to 5–10 million statements.

2.2.1 Installing for the First Time

Once you have downloaded a distribution, these steps will yield a functioning Knowledge Base (KB):

1. Extract the archive to your preferred location. On UNIX-like systems, `/usr/opt/ParliamentKB` or `/usr/local/ParliamentKB` is a likely choice. On Windows, `C:\Program Files\ParliamentKB` is customary. The instructions that follow refer to this directory as simply `ParliamentKB`.

²At present, only Intel/AMD architectures are supported. We expect to support Apple Silicon and ARM binaries in a future release.

2. On Windows, install the C and C++ run-time libraries. You can find installers for these in the following directory:

`ParliamentKB/RedistributablePackages`

3. On Windows, open a PowerShell, switch to the ParliamentKB directory, and invoke the startup script to start the Parliament server:

`.\parliament.ps1 -foreground`

For backwards compatibility, you may also use a command prompt and type:

`StartParliament.bat`

On other platforms, open a shell, switch to the ParliamentKB directory, and start Parliament using this command:

`./parliament foreground`

For backwards compatibility, you can also type:

`./StartParliament.sh`

Whatever platform you are using, this starts Parliament as a command line process running under the current terminal. You can shut down the server by typing “exit” or “Ctrl+C”.

The procedure above directly starts the Parliament’s server. However, in many cases it is preferable to run Parliament as a Windows service or as a Linux/UNIX daemon.

Installing as a Windows Service

Use the following procedure to install Parliament as a Windows service.

If you want Parliament to run under a dedicated account, start by creating that account. This step is optional.

Next create a data directory in a location of your choice. One reasonable option is `C:\ProgramData\parliament-data`. Parliament will store its content here, as well as logging and temporary files. Ensure that this directory is writable by the user established above. (Even if you did not create such a user, you may need to grant access to the “Local Service” account.)

Now unzip the Parliament distribution to a location of your choice, such as `C:\Program Files\ParliamentKB`. It is fine for this directory to be read-only for the user established above.

It is important to note that neither of the two locations above should be on a mapped network drive. This is because the mapping is an artifact of your user login, and services run outside that context.

Make the following configuration changes:

- In `parliament.ps1`, uncomment the two lines for the user name and password and fill in the user name and password established above, if you created one. If not, skip this step.
- Also in `parliament.ps1`, you may wish to edit certain other parameters, such as the host and port or the memory allocated to the JVM (see Section 2.3). This step is also optional.
- In `ParliamentKbConfig.txt`, set the `kbDirectoryPath` entry to the absolute path of the data directory you created above.

Finally, from the Parliament directory, issue this command in PowerShell:

```
.\parliament.ps1 -install
```

Then open the Services management console, select Parliament, and click the start button.

At this point, Parliament is running in the background, and whenever the system is shut down and restarted, Parliament will also properly shut down and restart. You can monitor and control (start, stop, and restart) Parliament via the Services management console as above.

To uninstall Parliament as a Windows service, shut it down and then run this command:

```
.\parliament.ps1 -uninstall
```

If you change any of the parameters in `parliament.ps1` after installing as a service, (see Section 2.3) or upgrade your Java installation, you will need to uninstall and re-install the service in order to have those changes reflected in the installed service definition.

Installing as a Linux Daemon

This section is focused on variants of Linux that support systemd, such as RHEL and Ubuntu. For setup on non-systemd Linux, please use the next section.

To install Parliament as a systemd service, we begin by establishing a system account (and group) under which Parliament will run. To do this from scratch, use the following command:

```
sudo adduser --system --group parliament    (Ubuntu)
sudo useradd --system -U parliament        (RHEL)
```

Next we need to create a data directory in a location of your choice, such as `/var/parliament-data`. Parliament will store its content here, as well as logging and temporary files. Ensure that this directory is owned and writable by the group and user established above:

```
sudo chmod u=rwx,go=rx /var/parliament-data
sudo chown -R parliament:parliament /var/parliament-data
```

Now unzip the Parliament distribution to a location of your choice, such as `/opt/ParliamentKB` or `/usr/local/ParliamentKB`. It is fine for this directory to be owned by root and read-only for the group and user established above.

Make the following configuration changes:

- In `parliament`, uncomment the line containing `DAEMON_USER` and set it to the name of the user created above.
- Also in `parliament`, you may wish to edit certain other parameters, such as the host and port or the memory allocated to the JVM (see [Section 2.3](#)). This step is optional.
- In `ParliamentKbConfig.txt`, set the `kbDirectoryPath` entry to the absolute path of the data directory you created above.

Finally, from the Parliament directory, issue these commands:

```
sudo ./parliament install
sudo systemctl start parliament
```

At this point, Parliament is running in the background, and whenever the system is shut down and restarted, Parliament will also properly shut down and restart. You control Parliament via the `systemctl` command:

```
sudo systemctl { start | stop | restart } parliament
```

You can check whether Parliament is running with either of these commands:

```
ps -ef | grep -i parliament | grep -v grep
sudo systemctl show parliament
```

To uninstall Parliament as a `systemd` service, run this command:

```
sudo ./parliament uninstall
```

Installing as a UNIX Daemon

For UNIX and non-`systemd` Linux variants, this command will run Parliament as a detached process suitable to run as a daemon:

```
./parliament { start | stop | restart }
```

However, there is currently no built-in support for wiring this into the operating system's infrastructure for starting and stopping daemon processes, because this infrastructure varies from platform to platform. In the future, this may be remedied for specific platforms, such as MacOS X.

2.2.2 Upgrading an Existing Installation

When upgrading an existing Parliament installation to a newer version, the procedures of Section 2.2.1 are generally not sufficient because they ignore the migration of the data. The Parliament development team strives to maintain compatibility of file formats whenever possible. In such cases, all that is required is to move the `kb-data` subdirectory of your Parliament installation into the new Parliament installation. Of course, if you have customized Parliament's configuration, you will also need to make those changes in the new installation as well.

However, from time to time Parliament's file formats do change, and in such cases a different migration procedure is required. Following this procedure even in those cases where it is not required has some benefits as

well, since it frees up any unused space in the Parliament data files. This is the way to perform such a migration:

1. **Before upgrading**, point your browser at your Parliament server. The URL for this is `http://<host>:<port>/parliament/`. By default this will be `http://localhost:8089/parliament/`.
2. Click on the “Export” link.
3. Click on the “Export Repository” button. This will start a download whose file name looks like this:

`parliament-export-localhost-<date>-<time>.zip`
4. When the download finishes, shut down Parliament. Rename the Parliament installation directory if you want your upgraded installation to reside in the same place.
5. Unzip the new version of Parliament to your desired location, customize the new installation’s settings as required (see Section 2.3), and start up the new Parliament installation.
6. Point your browser at the new Parliament server. The URL is the same as the one in Step 1 above.
7. Click on the “Insert Data” link. In the “Import Repository” section, press the “Choose File” button and select the file you downloaded in Step 3 above. Then press the “Import Repository” button.

Once you see the “Insert operation successful” message, your upgraded Parliament server is ready for business.

2.2.3 Usage

Shutting Parliament Down

Important note: When you shut down Parliament, it is important to shut it down gracefully. Otherwise, the Parliament files may not be flushed to disk before they are closed, and they may be corrupted. If you run Parliament as a Windows service or Linux/UNIX Daemon (see Section 2.2.1), this is generally not a problem, because the operating system sends a shutdown message at the appropriate times. However, if you run Parliament

explicitly using the command

<code>parliament foreground</code>	(on Linux/UNIX)
<code>.\parliament.ps1 -foreground</code>	(on Windows)

then you will need to shut it down yourself by typing the command “exit” followed by «return» or «enter».

Parliament Connection URLs

To use the Parliament server, you will need the the connection URLs listed in Table 2.3. (See Section 2.3 to configure your server to run on a host name and port other than “localhost” and “8089”.)

HTTP interface:	<code>http://localhost:8089/parliament</code>
SPARQL endpoint:	<code>http://localhost:8089/parliament/sparql</code>
Bulk loader:	<code>http://localhost:8089/parliament/bulk</code>

Table 2.3: Parliament Server Connection URLs

Parliament’s Web Interface

You can use the Parliament server interactively by pointing a web browser at the first URL in Table 2.3. This simple and (hopefully) self-explanatory web interface is useful for initial data loading, experimenting with queries, exploring a collection of data, troubleshooting, and similar tasks.

Using Parliament Programmatically

In order to issue queries programmatically, you will need to write some code to send SPARQL-compliant HyperText Transfer Protocol (HTTP) requests. One library that can send such requests on your behalf is Jena itself. Figure 2.1 shows sample code for issuing a SPARQL select query. Note that this code uses the SPARQL endpoint URL from Table 2.3.

Similarly, we can also update the content of Parliament programmatically using the Jena library to send SPARQL-Update requests. Figure 2.2 shows sample code for issuing an update.

```
import org.apache.jena.query.QueryExecution;
import org.apache.jena.query.QueryExecutionFactory;
import org.apache.jena.query.QuerySolution;
import org.apache.jena.query.ResultSet;

void issueSelectQuery(String sparqlSelectQuery) {
    try (QueryExecution exec = QueryExecutionFactory.sparqlService(
        "http://localhost:8089/parliament/sparql",
        sparqlSelectQuery)) {
        for (ResultSet rs = exec.execSelect(); rs.hasNext();) {
            QuerySolution qs = rs.next();
            // Do something useful with the query solution set here
        }
    }
}
```

Figure 2.1: Issuing a SPARQL select query with Jena 3

```
import org.apache.jena.update.UpdateExecutionFactory;
import org.apache.jena.update.UpdateFactory;
import org.apache.jena.update.UpdateRequest;

void issueUpdate(String sparqlUpdate)
{
    UpdateRequest request = UpdateFactory.create(sparqlUpdate);
    UpdateExecutionFactory
        .createRemote(request, "http://localhost:8089/parliament/sparql")
        .execute();
}
```

Figure 2.2: Issuing a SPARQL-Update request with Jena 3

Another common update operation is inserting a body of RDF into Parliament, shown in Figure 2.3.

```
import java.util.Iterator;
import org.apache.jena.graph.Node;
import org.apache.jena.rdf.model.Model;
import org.apache.jena.rdf.model.Statement;
import org.apache.jena.sparql.core.Quad;
import org.apache.jena.sparql.modify.request.QuadDataAcc;
import org.apache.jena.sparql.modify.request.UpdateDataInsert;
import org.apache.jena.update.UpdateExecutionFactory;
import org.apache.jena.update.UpdateRequest;

void issueInsert(Model model, Node graphUri)
{
    QuadDataAcc acc = new QuadDataAcc();
    for (Iterator<Statement> iter = model.listStatements(); iter.hasNext();) {
        Statement statement = iter.next();
        acc.addQuad(new Quad(graphUri, statement.asTriple()));
    }

    UpdateRequest updateRequest = new UpdateRequest(new UpdateDataInsert(acc));
    updateRequest.setPrefixMapping(model);
    UpdateExecutionFactory
        .createRemote(updateRequest, "http://localhost:8089/parliament/sparql")
        .execute();
}
```

Figure 2.3: Inserting RDF with Jena 3

Inserting Data into Parliament from the Command Line

Parliament's client-side library provides a handy command line utility for inserting a file containing RDF into a running Parliament instance. The command here is broken over several lines only to fit the printed page:

```
java -cp "clientJars/*"
com.bbn.parliament.client.jena.RemoteInserter
<hostname> <port> <inputfile> [<graph-name>]
```

where `clientJars` is the full path name of the directory of that name under your Parliament installation.

Using Parliament's RemoteModel Class

Parliament also provides a Java library to help programmers interact with the Parliament server, which can be found here:

ParliamentKB/lib/ParliamentClient.jar

This library leverages the Jena functionality shown above, but exposes a few extra features of the Parliament server that are not accessible within the bounds of SPARQL. Note, however, that for common operations, there is no need to use this library.

To use this approach, first add the above jar file to your class path. Then create a RemoteModel object as shown in Figure 2.4. This object allows you to access and manipulate the Parliament repository identified by the URLs passed to the constructor, which are again taken from Table 2.3. Note that the query strings passed to the remote model in Figure 2.4 fol-

```
import com.bbn.parliament.client.jena.RemoteModel;

void useRemoteParliamentModel()
{
    // Create model:
    RemoteModel rmParliament = new RemoteModel(
        "http://localhost:8089/parliament/sparql",
        "http://localhost:8089/parliament/bulk");

    // Insert contents of another Jena Model:
    rmParliament.insertStatements(myPopulatedJenaModel);

    // Perform SPARQL SELECT query:
    ResultSet results = rmParliament.selectQuery(myQueryString);

    // Add statements using SPARQL-UPDATE:
    rmParliament.updateQuery(mySparqlUpdateQueryString);
}
```

Figure 2.4: Using Parliament Via a Remote Jena Model

low standard SPARQL and SPARQL-UPDATE formats, including support for named graphs. Also note that the RemoteModel class exposes the bulk loading feature of the Parliament server, allowing the efficient loading of large bodies of RDF.

2.2.4 Securing a Parliament Server

This section discusses how to use an Apache web server (httpd) or NGINX on Linux as a secure proxy for a Parliament server running on the same machine. This procedure will implement TLS so that the communication channel is encrypted and the server is authenticated to clients. Client authentication is not covered at this time.

Before we begin, update the operating system. (Note that the commands below use yum. Please substitute apt for yum if your Linux distribution uses that package manager instead.)

```
sudo yum update
```

Next install required software.

```
sudo yum install «java-development-kit»
```

where «java-development-kit» is `java-8-amazon-corretto-headless` on AWS and `openjdk-8-jdk-headless` in other environments. Next install your desired Web server to serve as a proxy. To use Apache:

```
sudo yum install httpd mod_ssl
```

Or to use NGINX:

```
sudo yum install nginx-light
```

On your virtual machine, configure the following:

- Deploy Parliament in the standard way. (See Section [2.2.1](#) or [2.2.2](#) above for details.)
- It is crucial that in the startup script `parliament`, located in the root directory of your Parliament installation, the `JETTY_HOST` setting near the top *must* be set to `localhost` (or `127.0.0.1`). This prevents Parliament itself from being directly exposed to network traffic from outside the machine.
- In those same two files, ensure `JETTY_PORT` is set to `8089`. In reality, most any port may be used here. `8089` is Parliament's default, and a useful choice. The instructions below assume this port.
- Ensure that any firewalls (either on the machine itself or elsewhere)

are set to allow HTTP (port 80) and HTTPS (port 443) traffic to reach the machine from any network locations that need access to the Parliament server.

Acquire a web server certificate for the machine from your favorite certificate authority. This typically includes three files:

- The certificate itself, referred to as `certificate.crt` below.
- The private key, called `private.key` below.
- The root and intermediate certificate chain, called `ca-bundle.crt` below.

Securing Parliament with Apache

Create a directory for the three certificate files and store them there. The actual location is not important, but we will refer to it as `/etc/server-certs` below. The three files, and especially the private key, should be stored only at this location on the server to prevent unauthorized disclosure. A backup on a separate, secure machine is also a good idea. To finish securing the files, run these commands:

```
sudo chmod u=rw,go= /etc/server-certs/*
sudo chown root.root /etc/server-certs/*
```

Copy the following file from your Parliament installation's `conf` directory to `/etc/httpd/conf.d`:

```
parliament-redirect-httpd.conf
```

In that file, substitute your machine's host name (the host name that you used when you requested the certificate) in the indicated places, and be sure that the file contains the correct paths to the three certificate files and the correct port for Parliament. Then restart the web server to activate the new configuration with this command:

```
sudo systemctl restart httpd
```

Securing Parliament with NGINX

Create a directory for the three certificate files and store them there. The actual location is not important, but we will refer to it as `/etc/server-certs` below. The three files, and especially the private key, should be stored only at this location on the server to prevent unauthorized disclosure. A backup on a separate, secure machine is also a good idea.

Unlike Apache, NGINX requires that the certificate and certificate chain be combined into a single file, like so:

```
cat certificate.crt ca-bundle.crt > temp.crt
rm certificate.crt ca-bundle.crt
mv temp.crt certificate.crt
```

Note that the order in which the two files are combined is important — the certificate must come before the certificate chain. Finish securing the files with these commands:

```
sudo chmod u=rw,go= /etc/server-certs/*
sudo chown root.root /etc/server-certs/*
```

Copy the following file from your Parliament installation's `conf` directory to `/etc/nginx/sites-available`:

```
parliament-redirect-nginx.conf
```

In that file, substitute your machine's host name (the host name that you used when you requested the certificate) in the indicated places, and be sure that the file contains the correct paths to the two certificate files and the correct port for Parliament. From `/etc/nginx/sites-enabled`, create a symbolic link:

```
ln -s /etc/nginx/sites-available/parliament-redirect-nginx.conf
```

Finally, restart the web server to activate the new configuration with this command:

```
sudo systemctl restart nginx
```

2.3 Configuring Parliament

The section begins with a how-to for common configuration tasks, followed by a more detailed discussion of each of Parliament's configuration files.

2.3.1 Common Configuration Tasks

Changing the host name and port: See Section 2.3.2.

Changing the location of Parliament's data files: See the `kbDirectoryPath` setting in Section 2.3.3.

Setting the query timeout: See the `TimeoutDuration` and `TimeoutUnit` settings in Section 2.3.3.

Memory configuration: Parliament uses three separate pools of memory:

- The Java heap (to configure, see Section 2.3.2)
- The demand-paged cache maintained by the operating system for memory-mapped files (not configurable)
- The C++ heap (not configurable)

Achieving the right balance between these pools of memory is tricky, but here are a few guidelines:

- Don't make the Java heap too big. Many Java programmers are accustomed to setting their Java heap as large as possible, but for Parliament, the Java heap is not as important as the memory-mapped files.
- The Java heap size should be significantly less than the total memory available, in order to leave sufficient memory for the memory-mapped files.

Concurrency: Parliament has a many readers/single writer model of concurrency. In other words, if Parliament receives several requests at once, and all of them are read-only requests (such as a query), then Parliament will execute those requests in parallel. Whenever Parliament receives a request that requires writing (such as an insert, a SPARQL Update, or a graph creation), that request will wait until all other executing requests are finished, and then it will execute. And while it is executing, all other requests

(read or write) will wait in the queue until it is finished.

This model works well in most situations, where the number of concurrent requests is not terribly large and read-only requests dominate. A few dozen concurrent queries is not uncommon in our experience, and works well. Larger numbers of concurrent read-only requests will work correctly, but the performance may degrade because all those requests ultimately are reading from the same I/O device. Therefore, caching becomes the key issue for concurrency levels this high (see memory configuration above).

2.3.2 Parliament's Startup Script

Parliament's startup script, located in the root installation directory, contains a few configuration settings. On Windows it is the PowerShell script `parliament.ps1`, while other platforms use the bash script `parliament`.

Change the host name (or IP address) and port of the SPARQL endpoint by setting `JETTY_HOST` and `JETTY_PORT` in the startup script (`$jettyHost` and `$jettyPort` on Windows). These default to `localhost` and `8089`, respectively. To make Parliament addressable on any of a host's network interfaces, change `JETTY_HOST` to `"0.0.0.0"`.

Setting `DEFAULT_JAVA_HEAP_SIZE` (`$defaultJavaHeapSize` on Windows) controls the amount of memory set aside for the Java heap. A common mistake of people used to Java software is to set this to the total memory on the machine. While it is certainly important to give the Java Virtual Machine (JVM) sufficient memory, a significant portion of Parliament is native code that does not use the Java heap. Setting this parameter too high will starve the native code of the memory it needs. The default of 512 MB is reasonable for machines with 4 to 8 GB of total memory. On Windows, this value must be an integer in units of MB. On other platforms, it is an integer concatenated with an 'm' (MB) or 'g' (GB).

If you are running the server as a Windows service, you must re-run the startup script with the `install` option to make your changes effective.

2.3.3 Parliament's Main Configuration File

Parliament's main configuration file contains settings pertaining to storage, query, and inference. It is typically called `ParliamentKbConfig.txt`, and Parliament finds it like so:

1. If the environment variable `PARLIAMENT_KB_CONFIG_PATH` is set, its value is assumed to be the configuration file's absolute path. Note that using this option, you can rename the file to anything you wish.
2. If this environment variable is not set, then Parliament looks for a file named `ParliamentKbConfig.txt` in the same directory as the Parliament Dynamic Link Library (DLL) (on Windows) or shared library (on other platforms).
3. If that file is not found, then Parliament looks for a file of the same name in the parent directory of the Parliament DLL or shared library. In practice, this is typically how the configuration file is found.
4. Otherwise, Parliament looks for a file named `ParliamentKbConfig.txt` in the current working directory of the server process.

The configuration file is a plain text file containing name-value pairs. The names are case-insensitive. Blank lines and comment lines (preceded by a '#') are ignored. Many of the settings are Boolean values, in which case the value is case-insensitive and may be "true", "t", "yes", "y", "on", or "1" (for true), or "false", "f", "no", "n", "off", or "o" (for false).

The recognized settings are as follows:

kbDirectoryPath The path of the Parliament knowledge base files. If they do not yet exist, they will be created here. Log files will also be stored here in the `log` sub-directory. *Default:* `kb-data`

stmtFileName The name of the memory-mapped file containing records of statements. *Default:* `statements.mem`

rsrcFileName The name of the memory-mapped file that contains resource records. *Default:* `resources.mem`

uriTableFileName The name of the memory-mapped file containing resource strings (URIs and literals). *Default:* `uris.mem`

uriToIntFileName The name of the file containing the mapping from numeric resource identifiers to resource strings. This file is managed by RocksDB. *Default: u2i.db*

stmtToIdFileName This obsolete setting is now ignored. It indicated the name of a file that mapped tuples of numeric resource identifiers to numeric statement identifiers. This file was omitted by default, and was included only if the setting “keepDupStmtIdx” was turned on. The default value was `stmt2id.db`. If your configuration had “keepDupStmtIdx” turned off, then you can safely delete these two settings from your configuration file. If it was turned on, then delete the file named by this setting and then delete these two settings from your configuration file.

keepDupStmtIdx This obsolete setting is now ignored. If set to “yes”, Parliament would maintain an extra file (named by the “stmtToIdFileName” option). It was intended to be a performance optimization, but it rarely helped. It defaulted to “no”. If you had this turned off then you can simply delete this and the “stmtToIdFileName” options from your configuration. If it was turned on, then delete the file named by the “stmtToIdFileName” option and then delete these two settings from your configuration file.

readOnly When set to “yes”, prevents Parliament from changing the underlying storage files in any way. *Default: “no”*

fileSyncTimerDelay Parliament periodically flushes its underlying data files to disk to decrease the chances of a file corruption and to limit the amount of time required to shut down Parliament gracefully. This parameter is the time interval in milliseconds between flushes. Set it to zero to disable flushing the files to disk. This setting applies only to Parliament deployed as a server. Any other deployment will ignore this setting. *Default: “15000”*

initialRsrcCapacity The number of resources Parliament should allocate space for when creating a new KB. *Default: “300000”*

avgRsrcLen The average resource length (in characters) that Parliament should anticipate when allocating space in a new KB. *Default: “100”*

rsrcGrowthIncrement The number of resources by which Parliament

increases the resource table size when it runs out of space in the file. If this is less than one, then linear growth of the resource table is disabled, in which case `rsrcGrowthFactor` must be larger than one (see below). *Default: “600000”*

rsrcGrowthFactor The factor by which Parliament increases the size of the resource table when it runs out of space in the file. If this is less than or equal to one, then geometric growth of the resource table is disabled, in which case `rsrcGrowthIncrement` must be at least one (see above). Note that this is a decimal number that should be formatted according to your locale, e.g., “1.1” in the US or “1,1” in Europe. *Default: “0”*

initialStmtCapacity The number of statements Parliament should allocate space for when creating a new KB. *Default: “500000”*

stmtGrowthIncrement The number of statements Parliament adds to the statement table size when it runs out of space in the file. If this is less than one, then linear growth of the statement table is disabled, in which case `stmtGrowthFactor` must be larger than one (see below). *Default: “1000000”*

stmtGrowthFactor The factor by which Parliament increases the size of the statement table when it runs out of space in the file. If this is less than or equal to one, then geometric growth of the statement table is disabled, in which case `stmtGrowthIncrement` must be at least one (see above). Note that this is a decimal number that should be formatted according to your locale, e.g., “1.1” in the US or “1,1” in Europe. *Default: “0”*

bdbCacheSize This obsolete setting is now ignored.

normalizeTypedStringLiterals Instructs Parliament to convert typed string literals to plain literals, both upon insert and at query time, as mandated in RDF 1.1. In Parliament versions before the implementation of this option (versions 2.7.9 and prior), the behavior was as if the option were set to “no”. Therefore, if you are migrating a Parliament installation originally created with one of these older versions to a newer version, you must do one of two things:

1. Either before or after the upgrade, create an export of the entire

triple store. To do this, use the “Export Repository” option on the Export page of Parliament’s Web-based administrative interface. Then shut down Parliament and delete its kb-data (or data) directory. This is the location identified by the kbDirectoryPath setting in the configuration file. Then, after upgrading, import the backup using the “Import Repository” option on the Insert Data page of the administrative interface.

2. Set this option to “no”.

The first option is highly recommended. It requires some effort, but will bring your triple store into compliance with RDF 1.1 and deliver slightly better performance to boot. *Default: “yes”*

TimeoutDuration Sets the query execution timeout. *Default: “5”*

TimeoutUnit Sets the units of the query execution timeout. Valid values are “nanoseconds”, “microseconds”, “milliseconds”, “seconds”, “minutes”, “hours”, and “days”. *Default: “minutes”*

runAllRulesAtStartup Causes Parliament to evaluate each of the enabled rules at startup against the existing statements in the KB to see if any new statements should be materialized. Generally, this is unnecessary, because the rules are always evaluated against each statement as it is asserted. However, if some of the rules were initially disabled when statements were being asserted, and are then enabled, it may be necessary to turn this on for one startup of Parliament to ensure that the state of the KB is in sync with the rule settings. Alternately, in such a case this setting may be left off and the ParliamentAdmin tool may be run against the KB with the “guaranteeEntailments” option. Note that when this setting is turned on, KB the startup time may be lengthy. *Default: “no”*

enableSWRLRuleEngine Enables an additional rule engine that implements the Semantic Web Rule Language (SWRL) rule language. *Default: “no”*

SubclassRule Enables the following inference rules:

$$A \subset B \wedge B \subset C \Rightarrow A \subset C$$

$$X \in A \wedge A \subset B \Rightarrow X \in B$$

$$X \in \text{rdfs:Class} \Rightarrow X \subset X$$

$$X \in \text{owl:Class} \Rightarrow X \subset X$$

where $\subset$ means “sub-class of” and $\in$ means “of type”. *Default: “on”*

inferRdfsClass When both this setting and “SubclassRule” are turned on, this enables the following inference rule:

$$A \subset B \Rightarrow A \in \text{rdfs:Class} \wedge B \in \text{rdfs:Class}$$

where $\subset$ means “sub-class of” and $\in$ means “of type”. *Default: “off”*

inferOwlClass When both this setting and “SubclassRule” are turned on, this enables the following inference rule:

$$A \subset B \Rightarrow A \in \text{owl:Class} \wedge B \in \text{owl:Class}$$

where $\subset$ means “sub-class of” and $\in$ means “of type”. *Default: “off”*

inferRdfsResource When both this and the “SubclassRule” settings are turned on, the following inference rule is enabled:

$$A \subset B \Rightarrow A \subset \text{rdfs:Resource} \wedge B \subset \text{rdfs:Resource}$$

where $\subset$ means “sub-class of”. *Default: “off”*

inferOwlThing When both this setting and “SubclassRule” are turned on, this enables the following inference rule:

$$A \subset B \Rightarrow A \subset \text{owl:Thing} \wedge B \subset \text{owl:Thing}$$

where $\subset$ means “sub-class of”. *Default: “off”*

SubpropertyRule Enables the following inference rules:

$$P \sqsubset Q \wedge Q \sqsubset R \Rightarrow P \sqsubset R$$

$$P \sqsubset Q \wedge P(X, Y) \Rightarrow Q(X, Y)$$

where $\sqsubset$ means “sub-property of”. *Default: “on”*

DomainRule Enables the following inference rule:

$$\text{rdfs:domain}(P, C) \wedge P(X, Y) \Rightarrow X \in C$$

Default: “on”

RangeRule Enables the following inference rule:

$$\text{rdfs:range}(P, C) \wedge P(X, Y) \Rightarrow Y \in C$$

Default: “on”

EquivalentClassRule Enables the following inference rule:

$$\text{owl:equivalentClass}(A, B) \Rightarrow A \subset B \wedge B \subset A$$

where $\subset$ means “sub-class of”. *Default: “on”*

EquivalentPropRule Enables the following inference rule:

$$\text{owl:equivalentProperty}(P, Q) \Rightarrow P \sqsubset Q \wedge Q \sqsubset P$$

where $\sqsubset$ means “sub-property of”. *Default: “on”*

InverseOfRule Enables the following inference rule:

$$\text{owl:inverseOf}(P, Q) \wedge P(X, Y) \Rightarrow Q(Y, X)$$

Default: “on”

SymmetricPropRule Enables the following inference rule:

$$P \in \text{owl:SymmetricProperty} \wedge P(X, Y) \Rightarrow P(Y, X)$$

Default: “on”

FunctionalPropRule Enables the following inference rule:

$$P \in \text{owl:FP} \wedge P(Z, X) \wedge P(Z, Y) \Rightarrow \text{owl:sameAs}(X, Y)$$

where “FP” stands for “FunctionalProperty”. *Default: “on”*

InvFunctionalPropRule Enables the following inference rule:

$$P \in \text{owl:IFP} \wedge P(X, Z) \wedge P(Y, Z) \Rightarrow \text{owl:sameAs}(X, Y)$$

where “IFP” stands for “InverseFunctionalProperty”. *Default: “on”*

TransitivePropRule Enables the following inference rule:

$$P \in \text{owl:TransitiveProperty} \wedge P(X, Y) \wedge P(Y, Z) \Rightarrow P(X, Z)$$

Default: “on”

2.3.4 Parliament's Logging Configuration

These files configure logging for Parliament's Java code:

```
ParliamentKB/conf/log4j.foreground.properties
ParliamentKB/conf/log4j.daemon.properties
```

They are standard Log4J configuration files. The first applies when running Parliament from the command line, and the second is used when running as a daemon or service.

The file `ParliamentLogConfig.txt` controls native code logging. Its format is similar to `ParliamentKbConfig.txt`, and it is located in a similar way (see Section 2.3.3), using file name `ParliamentLogConfig.txt` and the environment variable `PARLIAMENT_LOG_CONFIG_PATH`. The recognized settings are:

logToConsole Indicates whether to log to the console. *Default: "no"*

logConsoleAsynchronous Indicates whether to log asynchronously to the console. *Default: "no"*

logConsoleAutoFlush Indicates whether console log entries should be flushed after each log operation. *Default: "yes"*

logToFile Indicates whether to log to a file. *Default: "yes"*

logFilePath The path of the log file. May include the following placeholders to add contextual information to the file name:

%N Cardinal number of the log file. A number between the % and the N will left-pad with zeros to that many places.

%Y Year

%m Month

%d Day

%H Hours

%M Minutes

%S Seconds

Default: `log/ParliamentNative%3N_%Y-%m-%d_%H-%M-%S.log` relative to the `kbDirectoryPath` setting in `ParliamentKbConfig.txt`

logFileAsynchronous Indicates whether to log asynchronously to the

file. *Default: “no”*

logFileAutoFlush Indicates whether file log entries should be flushed after each log operation. *Default: “yes”*

logFileRotationSize The maximum approximate size (in bytes) of the log file. When this value is exceeded, the log file will be rotated. *Default: 10 MB*

logFileMaxAccumSize The maximum accumulated size of rotated log files. When this value is exceeded, the oldest log file is deleted. *Default: 150 MB*

logFileMinFreeSpace The minimum amount of free space on the volume where log files are stored. When the free space dips below this value, old log files will be deleted so as to free up enough space. *Default: 100 MB*

logFileRotationTimePoint The time of day at which the log file should be rotated regardless of its size. Must be in the format “HH:MM:SS”. *Default: 2:00 AM*

logLevel The global logging level. Must be one of TRACE, DEBUG, INFO, WARN, and ERROR. *Default: “INFO”*

logChannelLevel Overrides the global logging level for a single channel. (Generally a channel is equivalent to a class in the source code.) The value of this setting takes the form

«channel-name» = «log-level»

where «log-level» takes one of the values listed for the global logging level above. This option may be repeated to override the level for multiple channels. *Default: None*

2.3.5 Parliament’s Servlet Container Configuration

The file `ParliamentKB/conf/jetty.xml` configures the Jetty servlet container. Generally, there is no need to modify this file.

2.4 Parliament-Specific Features

Parliament has a number of features beyond those implied by the RDF, OWL, and SPARQL standards, detailed in the sub-sections below.

2.4.1 Reserved Predicates

The `par:directType` and `par:directSubClassOf` predicates are treated specially by Parliament. They are reserved in the sense that they may not be used as the subject, predicate, or object of any statements inserted in Parliament. However, when used in queries they have special meanings.

In queries, `par:directType` is interpreted like `rdf:type` except that inferred statements are ignored. Similarly, `par:directSubClassOf` is the same as `rdf:subClassOf` except that it ignores inferred statements. These predicates provide high-performance shortcuts for queries that are looking for a “most-derived type.” For instance, to find the most-derived type of a resource `?x`, we can use the following query:

```
prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#>
select distinct ?x ?type where {
  ?x a ?type .
  filter not exists {
    ?x a ?type2 .
    ?type2 rdfs:subClassOf ?type .
    filter ( ?type2 != ?type )
  }
}
```

This operates by first finding all of `?x`’s types and then filtering those that have a distinct sub-type. This is correct and SPARQL-compliant, but the negation is expensive. Alternately, in Parliament we can write this:

```
prefix par: <http://parliament.semwebcentral.org/parliament#>
select distinct ?x ?type where {
  ?x par:directType ?type .
}
```

This is no longer standards-compliant, and it won’t necessarily produce exactly the same results. However, it is significantly less verbose and, in many cases, much faster.

2.4.2 Reification

[Yet to be written]

2.4.3 Configuring and Using Indexes

[Yet to be written]

2.5 Parliament in Other Servlet Containers

The most common way to deploy Parliament uses the Jetty servlet container. This scenario is discussed above in Section 2.2. However, the Parliament distribution contains a war file that can be deployed in any servlet container. This section discusses how to deploy Parliament in other servlet containers, such as Apache Tomcat.

1. First acquire and install Apache Tomcat³ according to its own installation instructions. This will result in a directory containing your Tomcat installation, which, in keeping with the customary nomenclature of Tomcat, we will call CATALINA_HOME here.
2. Create a directory for the native Parliament binaries. This can reside anywhere on your machine, but the directory CATALINA_HOME/bin/parliament is a good choice.
3. Copy the native binaries into this directory from the bin directory of the Parliament binary distribution.
4. Copy Parliament's configuration files ParliamentKbConfig.txt and ParliamentLogConfig.txt from the root of the Parliament distribution into the CATALINA_HOME/bin/parliament directory.
5. Copy parliament.war from the webapps directory of the Parliament distribution into Tomcat's webapps directory, which by default is the directory CATALINA_HOME/webapps.
6. On Windows only, install the run-time libraries by running the installer in the RedistributablePackages subdirectory of your Parliament distribution.

³<http://tomcat.apache.org/>

7. In your favorite text editor, open the file

```
CATALINA_HOME/bin/parliament/ParliamentKbConfig.txt
```

and change the following line:

```
kbDirectoryPath = kb-data
```

to point to a directory suitable for storing your knowledge base. In an operational deployment of Parliament, this is often set to a large drive dedicated to the storage of the knowledge base, such as a Redundant Array of Inexpensive Disks (RAID), Network-Attached Storage (NAS), or Storage Area Network (SAN). For testing purposes, the default is fine. This will store the knowledge base in CATALINA_HOME/bin/parliament/kb-data.

8. If you have not already done so while installing Tomcat, create a file called `setenv.sh` in CATALINA_HOME/bin and within it set the environment variable CATALINA_OPTS like so:

```
JAVA_HEAP_SIZE=512m
PMNT_BIN="$CATALINA_HOME/bin/parliament"
export CATALINA_OPTS=-Djava.library.path=$PMNT_BIN
export CATALINA_OPTS="$CATALINA_OPTS -Xmx$JAVA_HEAP_SIZE"
```

On Windows, the file name should be `setenv.bat` and its content should look like this:

```
set JAVA_HEAP_SIZE=512m
set PMNT_BIN=%CATALINA_HOME%\bin\parliament
set CATALINA_OPTS=-Djava.library.path=%PMNT_BIN%
set CATALINA_OPTS=%CATALINA_OPTS% -Xmx%JAVA_HEAP_SIZE%
```

Note that you can control the amount of memory set aside for the Java virtual machine by changing `JAVA_HEAP_SIZE` in the scripts above. While it is important to allow the JVM sufficient memory, it is also important to realize that Parliament is native code, and therefore does not use the Java heap. Java programmers are often inclined to set `JAVA_HEAP_SIZE` close to the total memory of the machine, but this will starve Parliament of the memory it needs to achieve good performance. The default values given above, 128 MB and 512 MB, are reasonable values for machines with 4 to 8 GB of total memory.

9. Start Tomcat according to its documentation. If you redeploy Parliament into a running Tomcat instance, the server will need to be restarted. For instructions on using your Parliament server, see Section 2.2.3.
10. *Please note:* When you want to shut down the KB, it is important to shut it down gracefully. Otherwise, the Parliament files may not be flushed to disk before they are closed, and they may be badly corrupted. If you run Tomcat as a Windows service or UNIX Daemon, this is generally not a problem, because the operating system sends a shut-down message at the appropriate time. If, however, you run Tomcat explicitly via its startup script, you will need to shut it down yourself with Tomcat's stop script.

Glassfish Notes

When deploying Parliament to Glassfish, note the following:

- Glassfish sets its own `java.library.path` property. The Parliament libraries must either be copied into `«Glassfish install dir»/lib` or you must alter the Glassfish configuration such that the libraries are available on `java.library.path`.
- By default, Parliament stores its data files in the `kb-data` subdirectory of the current working directory. In the case of Glassfish, this will be `«Glassfish install dir»/domains/domain1/config/kb-data`. To change this, edit `ParliamentKbConfig.txt` to change the setting `kbDirectoryPath` to the location (absolute path) where your data files reside.
- If Parliament is redeployed into a running instance of Glassfish, the server will need to be restarted.

2.6 Using Parliament In-Process

Jena is a popular toolkit for manipulating RDF data in Java programs. The central concept in the Jena class library is the Model interface. An object of type Model represents a graph of RDF data, and features many methods for manipulating that data. With just a few lines of code, you can create a

Jena Model that is backed by an instance of Parliament, as demonstrated by Figure 2.5. The `createParliamentModel` method returns a Jena model

```
import java.io.File;
import com.bbn.parliament.kb_graph.KbGraph;
import com.bbn.parliament.kb_graph.KbGraphFactory;
import com.bbn.parliament.core.jni.Config;
import com.hp.hpl.jena.rdf.model.Model;
import com.hp.hpl.jena.rdf.model.ModelFactory;

private Model createParliamentModel()
{
    KbGraph baseGraph = KbGraphFactory.createDefaultGraph();
    return ModelFactory.createModelForGraph(baseGraph);
}
```

Figure 2.5: Creating a Jena Model backed by Parliament

backed by a Parliament instance. With this model, you can do any of the things you normally do with a Jena model, such as call `read` on it to load a file into it, or query it.

The `useParliamentModel` method shown in Figure 2.6 demonstrates how to call `createParliamentModel`. The important thing to note here is the `try-finally` construct. This guarantees that the model is closed properly, even if an exception is thrown. Closing the model is crucial, because if you don't, the Parliament files will not be flushed to disk before they are closed, and they will almost certainly be badly corrupted.

Naturally, there is some setup required to make this work:

1. You will need `KbGraph.jar` and `Parliament.jar` from your binary distribution, along with the Jena jar files, which you can download from the Jena web site.⁴ Place all of these jar files in a convenient location and ensure that they are added to your class path.
2. Copy the following files from the `bin` directory of the Parliament distribution to a convenient location, such as the directory used for the jar files above:

⁴<https://jena.apache.org/>

```
import com.hp.hpl.jena.rdf.model.Model;

void useParliamentModel()
{
    Model kbModel = createParliamentModel();
    try
    {
        // Use the model according to the Jena documentation.
        // For instance, to issue a SPARQL query:
        QueryExecution exec = QueryExecutionFactory.create(
            sparqlQueryString, kbModel);
        for (ResultSet rs = exec.execSelect(); rs.hasNext(); rs.next())
        {
            // Do something useful with the result set here
        }
    }
    finally
    {
        if (kbModel != null && !kbModel.isClosed())
        {
            kbModel.close();
            kbModel = null;
        }
    }
}
```

Figure 2.6: Using a Jena Model backed by Parliament

- All of the DLLs.⁵
 - ParliamentAdmin (ParliamentAdmin.exe on Windows)
 - ParliamentKbConfig.txt
 - ParliamentLogConfig.txt
3. To ensure that the JVM can find the DLLs or shared libraries, add their directory to the Java system property `java.library.path`.
 4. On Windows only, install the run-time libraries by running the installer in the `RedistributablePackages` subdirectory of your Parliament distribution.
 5. Customize the configuration `ParliamentKbConfig.txt` as desired. In particular, the `kbDirectoryPath` setting defaults to `./kb-data`. To place your Parliament KB files in a different location, change this setting to the directory of your choice. This is especially useful for placing the KB files on a drive array or for maintaining several KBs and switching between them.

If you do not wish to use the Parliament configuration file to configure your application, you can replace the first line of the method in Figure 2.5, `createParliamentModel`, with a line that creates a `Config` instance using the default constructor and then set the `Config` fields yourself. This may be useful for centralizing your application's configuration in a single file.

If `createParliamentModel` throws an `UnsatisfiedLinkError` exception, see Section 2.8 for possible resolutions.

2.7 The ParliamentAdmin Utility

[Yet to be written]

2.8 Troubleshooting

Because Parliament is most often used within a Java environment, the most common problem is the dreaded “Unsatisfied Link Error”. This error is generated by the JVM when the Java code requests the loading of a DLL,

⁵Or “shared libraries” on UNIX-like systems. For brevity, we will just use “DLLs.”

but the JVM is unable to load that DLL. There are quite a few reasons why you may encounter an unsatisfied link error, and unfortunately, the JVM is not very good about generating an illuminating error message. So, should you encounter this problem, here are some things to keep in mind:

- Double-check that you have followed all the deployment instructions correctly for your chosen mode of deployment.
- Make sure that your system is up-to-date with respect to patches. For Windows, this means a visit to the Windows Update web site.⁶ Most other operating systems have a similar facility.
- Be sure that a 64-bit build of Parliament is loaded by a 64-bit version of Java, and that a 32-bit build of Parliament is loaded by a 32-bit version of Java.
- When the JVM loads the Parliament DLL, Parliament itself loads several other DLLs. Even if the JVM is able to load Parliament, an unsatisfied link error will result if Parliament cannot load one of its subsidiary DLLs. Furthermore, the error message accompanying the unsatisfied link error usually does not indicate which DLL caused the load failure. The subsidiary DLLs fall into the following three categories:
 - The Boost and RocksDB DLLs: In the deployment procedures detailed in this document, these DLLs should reside in the same location as Parliament itself.
 - The C and C++ run-time libraries: On Unix-like systems these should be available automatically. On Windows you may need to install the Visual Studio run-time libraries. You can find installers for these in the `RedistributablePackages` subdirectory of your binary distribution of Parliament.
 - Various operating system DLLs: These are rarely a problem, because the OS needs to be able to find these to function.
- The Java system property `java.library.path` is a list of paths that Java searches when loading native libraries. This suggests that all

⁶<http://windowsupdate.microsoft.com/>

of the operating system-specific rules for locating DLLs can be circumvented by providing a suitable definition for this system property. Unfortunately, when Java loads a library that itself loads other libraries (as Parliament does), Java can consult `java.library.path` only when loading the top-level library, because the loading process for its dependents is managed entirely within the operating system. Thus, this property does not solve the problem for Parliament.

- Running `ParliamentAdmin` helps to diagnose unsatisfied link errors, because this tool also loads the Parliament DLL, but it does so without involving Java (because `ParliamentAdmin` is written in C++). This technique can be used to isolate the error to either the operating system level (when `ParliamentAdmin` fails to load the DLL) or to the Java level (when `ParliamentAdmin` succeeds). Even the simple command “`ParliamentAdmin -v`” to get the version requires loading the Parliament DLL, so this is an easy test.
- Multiple Java installations can confuse the DLL loading process. If you invoke Parliament using any of the following:

```
parliament {foreground|install}    (on Linux/UNIX)
.\parliament.ps1 -foreground        (on Windows)
```

then the Java installation is found by simply invoking the command `java` and relying on the operating system to locate the installation. In these cases, the `JAVA_HOME` environment variable is not used, unless the operating system uses it.

On Windows, the process by which the `java` command finds the JVM is quite complex. The following web page sheds some light on how you can manage this process:

<http://mindprod.com/jgloss/javaexe.html#MULTIPLES>

The following startup commands find Java differently:

```
parliament {start|stop|restart}    (on Linux/UNIX)
.\parliament.ps1 -install           (on Windows)
```

Here the `java` command is not used at all, and instead the service executable directly loads the `jvm.dll` library, located via the `JAVA_HOME` environment variable. On Windows, this is done at the time the ser-

vice is installed, so if you upgrade your Java installation after installation, you will have to uninstall and reinstall the service in order for it to find the `jvm.dll` library.

- The process by which Parliament loads its DLLs on Windows is a bit complex:
 - The JVM finds `FixupParliamentPath.dll` using the Java system property `java.library.path`. (This works, because that DLL has no dependencies of its own.)
 - Code in `FixupParliamentPath.dll` adds the directory where it lives to the path.
 - Then the Parliament DLL is loaded. The JVM finds it using `java.library.path`, and the operating system finds its dependencies because they, `FixupParliamentPath.dll`, and the Parliament DLL all live in the same directory.
- On Mac OS X and Linux, the JVM finds the Parliament shared library by consulting the Java system property `java.library.path`, and the operating system looks for its dependencies via settings placed in the shared libraries during the Parliament build process. The command “`otool -L`” (on Mac) or `ldd` (on Linux) will show from where the operating system is trying to load subsidiary shared libraries.
- A debug build of Parliament will not work on Windows unless the corresponding version of Visual Studio is installed. This is because the debug versions of the C and C++ run-time libraries are included only with Visual Studio.
- More rarely, unsatisfied link errors can result from a bad build of Parliament that causes the symbols exported from the DLL to be named incorrectly. If this happen, the JVM will succeed in loading the DLL, but fail to find the entry points it needs. This results in an unsatisfied link error that is indistinguishable from the cases where the JVM cannot load the DLL at all. This condition is more likely to occur on Windows, and usually has something to do with the symbol `BUILDING_KBCORE` not being defined on the compiler command line.

Chapter 3

Building Parliament™

Parliament is a cross-platform, mixed-language library. It's core is written in portable C++, but it also has a Java interface. As a result of both the cross-platform and multi-language requirements, the build infrastructure for Parliament requires a little bit of work to configure. This chapter is your guide through that process.

Parliament's build infrastructure has two main parts. The top-level portion is based on Gradle, a build tool used in the Java development community. This portion of the infrastructure builds the Java half of the Parliament code base, and it also invokes the second portion, which is based on Boost.Build. Boost.Build is a system that is well-adapted to building C++ code. It has the advantages of being portable and much simpler to use than make files. It is also the standard build system of the Boost project, whose libraries are used by the C++ portion of Parliament.

This chapter will step through the libraries and tools that Parliament depends upon and show you how to configure them on your system. At the end of this chapter, you should have a working copy of the Parliament git repository from which you can build the Parliament binaries.

3.1 Platforms and Prerequisites

You will need an appropriate C++ compiler for each operating system on which you wish to build. Parliament has been tested on the platform and

compiler combinations shown in Table 3.1. The last column shows the corresponding Boost.Build toolset name, which will appear in the sections below as we configure the Parliament build infrastructure.

Platform	Compiler	Toolset
Windows 11	Visual Studio 2022 (Visual C++ 14.3)	msvc
Mac OS X 26 Tahoe	Xcode 26.5	clang
Ubuntu 24	GCC 14.2	gcc
RHEL 10	GCC 14.2	gcc

Table 3.1: Supported Platforms and Compilers

On Macintosh, Parliament builds as a universal binary (for both Apple Silicon and Intel chips) using Apple’s Xcode development tools.

Parliament requires the Java Developer Kit (JDK), version 25:

- On Windows and Mac OS, download the JDK installer from Oracle.¹
- On Linux, you may need to install one or more packages, depending on your distribution. On Ubuntu, the package you need is “openjdk-25-jdk-headless.”

Finally, you need a client for the git version control system.²

Once you have these prerequisites installed, you can clone the Parliament code base from here:

<https://github.com/raytheonbbn/parliament>

Because GitHub limits repository size, this does not include larger binary files that are required by the build. To supply these, download this archive:

<https://github.com/raytheonbbn/parliament/releases/download/dependencies-«latest-date»/parliament-dependencies-«latest-date».zip>

and expand it in the dependencies subdirectory of your clone. (You can find the latest date by viewing the Releases page for Parliament on GitHub.)

¹<https://www.oracle.com/java/technologies/>

²<https://git-scm.com/downloads>

3.2 Building RocksDB

Parliament uses RocksDB,³ an embedded persistent key-value store from Meta. RocksDB is dual-licensed under both the GPLv2 and Apache 2.0 Licenses. The following procedures are based on RocksDB version 11.1.1.

3.2.1 Building RocksDB on Windows

On Windows, we use Microsoft's `vcpkg` package manager to download and create the RocksDB library and its dependencies. In your working copy, change to the `rocksdb-windows` directory. If you are running Windows on an Intel- or AMD-based machine, issue these commands from a Visual Studio terminal:

```
vcpkg install --triplet x64-windows
vcpkg install --triplet x64-windows-static
vcpkg install --triplet x86-windows
vcpkg install --triplet x86-windows-static
```

This takes a while to execute, so maybe pop a pizza in the oven.

If you are running Windows on an ARM-based machine, issue these commands instead:

```
vcpkg install --triplet arm64-windows
vcpkg install --triplet arm64-windows-static
```

[Something about setting up environment variables or `site-config.jam`.]

Start the Developer Command Prompt window that is a part of Visual Studio installation. Run the build commands from within the Developer Command Prompt window to have paths to the compiler and runtime libraries set.

3.2.2 Building RocksDB on Macintosh

On Mac OS, RocksDB and its dependencies are installable via HomeBrew. However, Parliament wants universal binaries, which we need to build ourselves. This section details alternate instructions to create RocksDB li-

³<https://rocksdb.org/>

libraries for use with Parliament, starting with RocksDB's dependencies. To build RocksDB on Macintosh, first be sure you are using the latest version of Xcode (since RocksDB requires C++20 support) and install CMake.⁴

Building Snappy

Snappy⁵ is an optional dependency of RocksDB. Because its build infrastructure cannot support universal binaries, we omit it.

Building GFlags

Download GFlags⁶ version 2.3.0 and then execute these commands:

```
tar xf gflags-2.3.0.tar.gz
cd gflags-2.3.0
cmake -S . -B build-cmake -DCMAKE_INSTALL_PREFIX=/opt/gflags-2.3.0 \
    -DBUILD_SHARED_LIBS=ON -DBUILD_STATIC_LIBS=ON \
    -DCMAKE_OSX_ARCHITECTURES="x86_64;arm64"
cd build-cmake
make
sudo make install
```

Building LZ4

Download LZ4⁷ version 1.10.0 and then execute these commands:

```
tar xf lz4-1.10.0.tar.gz
cd lz4-1.10.0
make PREFIX=/opt/lz4-1.10.0 CFLAGS="-arch x86_64 -arch arm64" \
    LDFLAGS="-arch x86_64 -arch arm64 -dynamiclib"
sudo make install PREFIX=/opt/lz4-1.10.0
```

Building ZStd

Download ZStd⁸ version 1.5.7 and then execute these commands:

⁴<https://cmake.org/>
⁵<https://github.com/google/snappy>
⁶<https://gflags.github.io/gflags/>
⁷<https://github.com/lz4/lz4>
⁸<https://github.com/facebook/zstd>

```
untar xf zstd-1.5.7.tar.gz
cd zstd-1.5.7
make PREFIX=/opt/zstd-1.5.7 CFLAGS="-arch x86_64 -arch arm64" \
    LDFLAGS="-arch x86_64 -arch arm64 -dynamiclib"
sudo make install PREFIX=/opt/zstd-1.5.7
```

Building RocksDB Itself

Define the following environment variables so that the builds can find the headers and libraries:

```
export GFLAGS_HOME=/opt/gflags-2.3.0
export LZ4_HOME=/opt/lz4-1.10.0
export ROCKSDB_HOME=/opt/rocksdb-11.1.1
export ZSTD_HOME=/opt/zstd-1.5.7
```

Now download and expand the source code archive for RocksDB version 11.1.1 from the releases page of the RocksDB GitHub site.⁹ Change line 1541 of `util/compression.cc` from this:

```
Status ExtractUncompressedSize(Args& args) override {
```

to this:

```
Status ExtractUncompressedSize([[maybe_unused]] Args& args) \
    override {
```

On lines 292, 307, 317, 327, 340, 353, 366, and 381 of the file `build_tools/build_detect_platform`, replace “-x” with “-c -x”. In addition, on line 381 replace “-o /dev/null” with “-o test.o”.

Comment out lines 355, 356, 360, 363, and 364 of `Makefile` by prefixing them with #.

Execute these commands:

```
make static_lib shared_lib PREFIX=/opt/rocksdb-11.1.1 \
    SKIP_DEPENDS=1 EXTRA_CXXFLAGS="-arch x86_64 -arch arm64 \
    -I${GFLAGS_HOME}/include -I${LZ4_HOME}/include \
    -I${ZSTD_HOME}/include" EXTRA_LDFLAGS="-arch x86_64 -arch arm64 \
    -dynamiclib -L${GFLAGS_HOME}/lib -L${LZ4_HOME}/lib \
    -L${ZSTD_HOME}/lib"
```

⁹<https://github.com/facebook/rocksdb/releases>


```
sudo make install PREFIX=/opt/rocksdb-11.1.1 SKIP_DEPENDS=1
```

Finally, run this test on the resulting dynamic library to be sure the build worked correctly:

```
otool -L $ROCKSDB_HOME/lib/librocksdb.11.1.1.dylib
```

This should result in output like the following:

```
.../librocksdb.11.1.1.dylib (architecture x86_64):
    librocksdb.10.10.dylib ...
    @rpath/libgflags.2.3.dylib ...
    /usr/lib/libz.1.dylib ...
    /usr/lib/libbz2.1.0.dylib ...
    /opt/lz4-1.10.0/lib/liblz4.1.dylib ...
    /opt/zstd-1.5.7/lib/libzstd.1.dylib ...
    /usr/lib/libc++.1.dylib ...
    /usr/lib/libSystem.B.dylib ...
.../librocksdb.11.1.1.dylib (architecture arm64):
    librocksdb.10.10.dylib ...
    @rpath/libgflags.2.3.dylib ...
    /usr/lib/libz.1.dylib ...
    /usr/lib/libbz2.1.0.dylib ...
    /opt/lz4-1.10.0/lib/liblz4.1.dylib ...
    /opt/zstd-1.5.7/lib/libzstd.1.dylib ...
    /usr/lib/libc++.1.dylib ...
    /usr/lib/libSystem.B.dylib ...
```

The main points to check for are (a) that the dynamic library contains two architectures (i.e., it is a universal binary) and (b) that all of the optional dependencies (GFlags, LibZ, LibBZ2, LZ4, and ZStd) are linked in.

3.2.3 Building RocksDB on Linux

To build RocksDB on Linux, upgrade your gcc compiler to at least version 11 to get C++20 support. Next, use the package manager to install RocksDB's dependencies. On Ubuntu, this looks like so:

```
sudo apt install libgflags-dev libsnappy-dev zlib1g-dev libbz2-dev \
    liblz4-dev libzstd-dev libxxhash-dev
```

Now execute the following commands from the root directory of the ex-

panded RocksDB release:

```
make static_lib shared_lib PREFIX=/opt/rocksdb-11.1.1 \  
    SKIP_DEPENDS=1  
sudo make install PREFIX=/opt/rocksdb-11.1.1 SKIP_DEPENDS=1
```

Define the following environment variable so that the Parliament build can find the headers and libraries:

```
export ROCKSDB_HOME=/opt/rocksdb-11.1.1
```

Finally, run this test on the resulting dynamic library to be sure the build worked correctly:

```
ldd $ROCKSDB_HOME/lib/librocksdb.so.11.1.1
```

This should result in output like the following:

```
/lib/ld-linux-aarch64.so.1 ...  
libbz2.so.1.0 ...  
libc.so.6 ...  
libgcc_s.so.1 ...  
libgflags.so.2.2 ...  
liblz4.so.1 ...  
libm.so.6 ...  
libsnappy.so.1 ...  
libstdc++.so.6 ...  
libxxhash.so.0 ...  
libz.so.1 ...  
libzstd.so.1 ...  
linux-vdso.so.1 ...
```

The main point to check is that all of the optional dependencies (GFlags, LibZ, LibBZ2, LZ4, and ZStd) are linked in.

3.3 Building the Boost Libraries

Download the Boost source distribution (version 1.91.0 or later) from the Boost web site. Unpack this to a handy location on your disk and define this environment variable pointing to that location:

```
export BOOST_ROOT=~/.boost_1_91_0    (on Mac OS and Linux)  
set BOOST_ROOT=C:\boost_1_91_0      (on Windows)
```

From Parliament's point of view, there are two primary components contained within `B00ST_ROOT`. The first (and most obvious) is the boost libraries themselves. Most of these are so-called "header-only" libraries, meaning that there is no pre-compiled library code. All of the code of such libraries is referenced via `#include` directives and compiled along with the calling code. Such libraries are extremely convenient, because they require little setup. Parliament also uses several Boost libraries that are not header-only. We will discuss how to build these libraries below.

The second major Boost component is `Boost.Build`. This is a cross-platform build system (located in the `B00ST_ROOT/tools/build`) that is written in a specialized interpreted language whose interpreter is a command called `b2`. The Boost community does not provide `b2` binaries. Rather, the Boost distribution contains a bootstrapping script that builds `b2` from source on your platform.

To build the minimal set of libraries required for Parliament, follow the directions below for your platform. In each case, you will invoke the bootstrap script to create the `Boost.Build` interpreter, and then you will invoke that to build the libraries.

3.3.1 Building Boost on Windows

First, identify (or create) a directory where we will place the `b2` build utility and define these environment variables:

```
set B2_HOME=«your-directory-here»
set PATH=%PATH%;%B2_HOME%\b2
```

Open a Command Prompt, change to the `B00ST_ROOT` directory, and issue these commands to build `b2`:

```
pushd tools\build
rd /s/q "%B2_HOME%\b2"
.\bootstrap.bat msvc & .\b2 install --prefix="%B2_HOME%\b2"
popd
```

On Intel-based computers, issue this command to build the boost libraries:

```
b2 -q --disable-icu --ignore-site-config --layout=versioned
--build-dir=build-msvc --stagedir=stage-msvc --with-atomic
```

```
--with-chrono --with-container --with-date_time --with-filesystem
--with-log --with-regex --with-serialization --with-test
--with-thread define=BOOST_LOG_USE_STD_REGEX
define=BOOST_LOG_WITHOUT_EVENT_LOG define=BOOST_LOG_WITHOUT_IPC
define=BOOST_LOG_WITHOUT_ASIO define=BOOST_LOG_WITHOUT_DEBUG_OUTPUT
define=BOOST_LOG_WITHOUT_SYSLOG
define=BOOST_TEST_ALTERNATIVE_INIT_API toolset=msvc
address-model=64 variant=debug,release link=shared,static
runtime-link=shared architecture=x86 cxxstd=20 stage
```

If you are using Windows on an ARM-based computer, use the same command, but change `architecture=x86` to `architecture=arm`.

This will build the libraries in `stage-msvc/lib`. The following commands will delete intermediate build products to save disk space:

```
rd /s/q build-msvc
del tools\build\b2.exe tools\build\src\engine\b2.exe
```

3.3.2 Building Boost on Macintosh

First, set these environment variables:

```
export B2_HOME=/opt/b2
export PATH=$B2_HOME/bin:$PATH
```

Open a shell, change to the `BOOST_ROOT` directory, and issue these commands to build `b2`:

```
pushd tools\build
./bootstrap.sh ; ./b2 install --prefix=temp-boost-build
sudo rm -r /opt/b2 ; sudo cp -R temp-boost-build /opt/b2
popd
```

Now issue this command to build the boost libraries:

```
b2 -q --disable-icu --ignore-site-config --layout=versioned
--build-dir=build-clang --stagedir=stage-clang --with-atomic
--with-charconv --with-chrono --with-container --with-date_time
--with-filesystem --with-log --with-regex --with-serialization
--with-test --with-thread define=BOOST_LOG_USE_STD_REGEX
define=BOOST_LOG_WITHOUT_SYSLOG define=BOOST_LOG_WITHOUT_IPC
define=BOOST_LOG_WITHOUT_ASIO
```

```
define=B00ST_TEST_ALTERNATIVE_INIT_API toolset=clang  
address-model=64 variant=debug,release link=shared,static  
runtime-link=shared architecture=arm+x86 cxxstd=20 stage
```

This will build the libraries in stage-clang/lib. The following command will delete intermediate build products to save disk space:

```
rm -r tools/build/b2 tools/build/src/engine/b2  
tools/build/temp-boost-build build-clang
```

3.3.3 Building Boost on Linux

First, set these environment variables:

```
export B2_HOME=/opt/b2  
export PATH=$B2_HOME/bin:$PATH  
export LINUX_DISTRO=«platform»
```

where «platform» indicates the particular flavor of Linux you are using. For the flavors of Linux that are officially supported, this should be set as given in Table 3.2.

<i>Platform</i>	<i>Description</i>
rhel10	Red Hat Enterprise Linux 10
ubuntu26	Ubuntu Linux 26, LTS
ubuntu24	Ubuntu Linux 24, LTS

Table 3.2: Supported Linux Flavors

Open a shell, change to the B00ST_R00T directory, and issue these commands to build b2:

```
pushd tools\build  
./bootstrap.sh ; ./b2 install --prefix=temp-boost-build  
sudo rm -r /opt/b2 ; sudo cp -R temp-boost-build /opt/b2  
popd
```

On Intel-based computers, issue this command to build the boost libraries:

```
b2 -q --disable-icu --ignore-site-config --layout=versioned  
--build-dir=build-msvc --stagedir=stage-msvc --with-atomic  
--with-chrono --with-container --with-date_time --with-filesystem
```

```
--with-log --with-regex --with-serialization --with-test
--with-thread define=BOOST_LOG_USE_STD_REGEX
define=BOOST_LOG_WITHOUT_EVENT_LOG define=BOOST_LOG_WITHOUT_IPC
define=BOOST_LOG_WITHOUT_ASIO define=BOOST_LOG_WITHOUT_DEBUG_OUTPUT
define=BOOST_LOG_WITHOUT_SYSLOG
define=BOOST_TEST_ALTERNATIVE_INIT_API toolset=msvc
address-model=64 variant=debug,release link=shared,static
runtime-link=shared architecture=x86 cxxstd=20 stage
```

If you are using Linux on an ARM-based computer, use the same command, but change `architecture=x86` to `architecture=arm`.

This will build the libraries in `stage-gcc-$LINUX_DISTRO/lib`. Run these commands to delete intermediate build products and save disk space:

```
rm -r tools/build/b2 tools/build/src/engine/b2
tools/build/temp-boost-build build-gcc-$LINUX_DISTRO
```

3.4 Configuring Boost.Build

Next we need to configure Boost.Build so that it can build Parliament. Start by defining the following environment variables:

- `LINUX_DISTRO`: (*Linux only*) The flavor of Linux you are using. See Table 3.2 in Section 3.3.3 above.
- `JAVA_HOME`: The location of your JDK installation, which must be version 25 or higher.
- `BOOST_VERSION`: The version of Boost (currently 1_91_0).
- `BOOST_ROOT`: As described above in Section 3.3.
- `BOOST_BUILD_PATH`: The sub-directory `tools/build` of `BOOST_ROOT`.
- `BOOST_TEST_LOG_LEVEL`: Controls the output volume of the C++ unit tests. Possible values and their meanings are listed in Table 3.3.

On Windows only, define the following environment variables as well:

```
set VS_ROOT=C:\Program Files\Microsoft Visual Studio\2022\Community\VC\
set VS_BAT_PATH_X86_64=!VS_ROOT!Auxiliary\Build\vcvars64.bat
set VS_BAT_PATH_ARM_64=!VS_ROOT!Auxiliary\Build\vcvarsarm64.bat
```

Value	Meaning
all	report all log messages
success	the same as all
test_suite	show test suite messages
message	show user messages (<i>useful default</i>)
warning	report warnings issued by user
error	report all error conditions
cpp_exception	report uncaught C++ exceptions
system_error	report system-originated non-fatal errors
fatal_error	report only fatal errors
nothing	do not report any information

Table 3.3: Possible Values of BOOST_TEST_LOG_LEVEL

```
set VS_REDIST_PATH_X86_64=!VS_ROOT!Redist\MSVC\v143\vc_redist.x64.exe
set VS_REDIST_PATH_ARM=!VS_ROOT!Redist\MSVC\v143\vc_redist.arm64.exe
```

Note that on Windows, in a command prompt, you will want to run one of the VS_BAT_PATH_«arch»_64 batch files to set up your command prompt for running the C++ compiler.

Next we create two Boost.Build configuration files, `site-config.jam` and `user-config.jam`. Boost.Build reads these files on startup. These files can be placed in a number of locations. Table 3.4 explains where Boost.Build searches to find these files.

OS	site-config.jam	user-config.jam
Unix-like:	/etc \$HOME \$BOOST_BUILD_PATH	\$HOME \$BOOST_BUILD_PATH
Windows:	%SystemRoot% %HOMEDRIVE%%HOMEPATH% %HOME% %BOOST_BUILD_PATH%	%HOMEDRIVE%%HOMEPATH% %HOME% %BOOST_BUILD_PATH%

Table 3.4: Boost.Build Search Paths for Configuration Files

Some people prefer to keep these files together with their Boost.Build in-

stallation, in `BOOST_BUILD_PATH`. However, there are example files in that directory already, so you will have to rename them if you choose this option. Others prefer to separate the configuration files from the Boost.Build installation so that they can update to a new version of Boost.Build without having to first save `site-config.jam` and `user-config.jam` and then restore them after the update is complete. On Windows, using the `HOME` location requires defining the environment variable `HOME`, because Windows does not define it by default.

Within your working copy of the Parliament repository, in the directories `doc/MacOS`, `doc/Windows`, and `doc/Linux`, you will find example configuration files for Macintosh, Windows, and Linux respectively that you can copy and customize. The most important customization that you need to make is to remove (or comment out) any lines in `user-config.jam` for compiler versions that you have not installed.

3.5 Building Parliament Itself

The file `build.properties`, located in the root of your working copy, controls the Parliament build. This file does not exist by default. If the build does not find it, it uses `build.properties.default` instead. The latter contains the build options used to create an official release of Parliament, which may include several different release builds. To build a single variant, first copy `build.properties.default` to `build.properties`. Then customize the latter. The file itself contains instructions. Please change `build.properties.default` directly *only if you intend to update the official release build options*.

This next bit about the top-level build doesn't work just yet, but should be working when we get to the next release.

You are now ready to build Parliament. From the root directory of your Parliament working copy, issue this command:

```
./gradlew build          (on Mac OS and Linux)
.\gradlew.bat build      (on Windows)
```

This builds the entire repository, including both native and Java code, and creates a distribution-ready package in this directory:

target/parliament-**x.y.z-platform**

where **x.y.z** is the Parliament version number and **platform** indicates the platform on which the distribution runs, as shown in Table 2.2.

To delete all build products, issue this command:

```
./gradlew clean      (on Mac OS and Linux)
.\gradlew.bat clean  (on Windows)
```

For more targeted builds, gradle can be run from the sub-projects in your working copy. These include (in the order they should be built):

- **Parliament**: The native code at the heart of Parliament, plus its Java Native Interface (JNI) interface
- **jena/ParliamentClient**: A Parliament-specific client-side Java library for communicating with a Parliament SPARQL endpoint
- **jena/SparqlQueryBuilder**: A client-side Java library to both issue queries and to dynamically build and modify queries at run time. This library is Parliament-independent.
- **jena/OntologyDrivenDataAccess**: A client-side Java library that issues queries and returns the results in the form of Java data objects. This library is Parliament-independent.
- **jena/OntologyBundle**: A Gradle plugin to package (bundle) an OWL ontology in a jar in order to make it easy to use in Java-based software projects.
- **jena/KbGraph**: Enables Jena to use Parliament for storing models
- **jena/NumericIndex**: An add-on to KbGraph for processing numeric queries efficiently
- **jena/TemporalIndex**: An add-on to KbGraph for processing temporal queries efficiently
- **jena/SpatialIndex**: An add-on to KbGraph for processing spatial queries efficiently
- **server**: A Spring-based SPARQL endpoint implemented on top of KbGraph.

When working on the native code portions of Parliament, it can be useful to run the `b2` portion of the build directly. To do so, change directory to `KbCore` (for the Parliament DLL itself), `AdminClient` (for the command line interface to Parliament), or `Test` (for the unit tests). These directories are located within the `Parliament` sub-directory of your working copy. Then issue the command

```
b2 -q «build-options»
```

Here «build-options» is a placeholder for one set of build options from `build.properties`, described above. The `-q` option causes `b2` to quit immediately whenever an error occurs.

Appendix A

Glossary

BBN RTX BBN Technologies

DAML DARPA Agent Markup Language

DARPA Defense Advanced Research Projects Agency

DLL Dynamic Link Library (Often used synonymously with “shared library” or “shared object.”)

HTTP HyperText Transfer Protocol

JNI Java Native Interface (A facility within the Java platform to allow Java code to directly call native code.)

JVM Java Virtual Machine

KB Knowledge Base

NAS Network-Attached Storage

OWL Web Ontology Language

Parliament Parliament™ (BBN’s triple store, so named because “parliament” is the collective noun for a group of owls. A triple store is a specialized database tuned to the unique needs of the Semantic Web data representation.)

RAID Redundant Array of Inexpensive Disks

RDF Resource Description Framework

RDFS RDF Schema

SAN Storage Area Network

SPARQL SPARQL Protocol and RDF Query Language (If this seems confusing, it is because SPARQL is a recursive acronym.)

SWRL Semantic Web Rule Language

URL Uniform Resource Locator (The address of a page on the World Wide Web, typically starting with the prefix “http:” or “https:”.)

W3C World Wide Web Consortium

Appendix B

Bibliography

- [1] Robert Battle and Dave Kolas. “Enabling the Geospatial Semantic Web with Parliament and GeoSPARQL”. In: *Semantic Web* 3.4 (Oct. 2012), pp. 355–370. URL: http://www.semantic-web-journal.net/sites/default/files/swj176_2.pdf (cit. on p. 3).
- [2] Dave Kolas. *Spatiotemporal Indexing in Parliament with Jena*. Report. Jena Users Workshop, 2010 Semantic Technology Conference, San Francisco, CA: RTX BBN Technologies, June 23, 2010. URL: http://asio.bbn.com/2010/06/semtech/Parliament_Spatiotemporal_Indexing.pdf (cit. on p. 3).
- [3] Dave Kolas, Ian Emmons, and Mike Dean. “Efficient Linked-List RDF Indexing in Parliament”. In: *Proceedings of the Fifth International Workshop on Scalable Semantic Web Knowledge Base Systems (SSWS 2009)* (Washington, DC). Lecture Notes in Computer Science 5823. Springer, Oct. 2009, pp. 17–32. URL: <http://ceur-ws.org/Vol-517/> (cit. on p. 2).